

滑动窗口解法详解

使用LeetCode 370为例

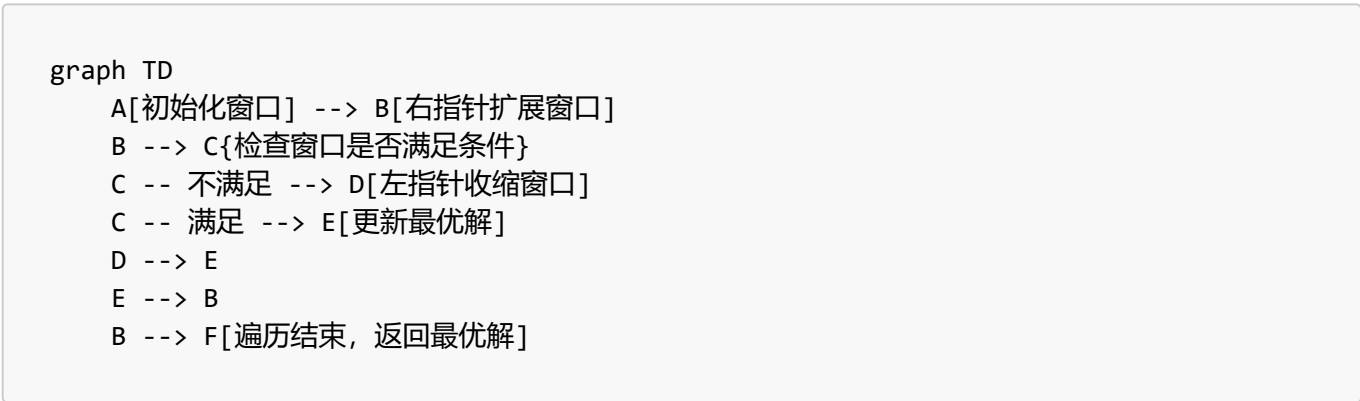
滑动窗口 (Sliding Window) 是一种**双指针**的进阶应用，核心思想是用两个指针 (左/右指针，也叫慢/快指针) 维护一个"窗口"，通过动态调整窗口的左右边界，在一次遍历中找到满足条件的最优解，时间复杂度通常为 $O(n)$ ，比暴力枚举 ($O(n^2)$) 高效得多。

1. 问题背景

题目要求：给定一个字符串 `s`，找出其中最多包含**两个不同字符**的最长子串的长度。比如输入 `s = "eceba"`，输出是 3 (最长子串是 "ece")；输入 `s = "ccaabbb"`，输出是 5 (最长子串是 "aabbb")。

2. 滑动窗口的核心逻辑

滑动窗口解决这类"子串/子数组满足某条件的最值"问题，通常遵循以下通用步骤：



具体到本题，我们拆解代码的每一步逻辑：

步骤1: 边界处理

```
if(s.length() < 3) return s.length();
```

如果字符串长度小于3，最多只有2个不同字符，直接返回长度即可（提前剪枝，减少无效计算）。

步骤2: 初始化核心变量

```
// 哈希表：记录窗口内每个字符的出现次数（键=字符，值=次数）
HashMap<Character, Integer> map = new HashMap<>();
int slow = 0; // 窗口左边界（慢指针）
int maxLen = 0; // 记录最长子串长度
```

- `map` 的作用：快速判断窗口内**不同字符的数量** (`map.size()` 就是不同字符数)；
- `slow`：窗口左边界，只有当窗口不满足条件时 (不同字符>2) 才向右移动；

- **fast**: 窗口右边界，遍历字符串时持续向右扩展。

步骤3: 右指针扩展窗口

```
for(int fast = 0; fast < s.length(); fast++){
    char c = s.charAt(fast);
    // 将当前字符加入窗口，更新计数（不存在则默认0，再加1）
    map.put(c, map.getOrDefault(c, 0) + 1);
}
```

fast 逐个遍历字符，把每个字符加入窗口，并记录其出现次数。

步骤4: 左指针收缩窗口（当窗口不满足条件时）

```
// 关键：当窗口内不同字符数>2时，收缩左边界
while(map.size() > 2){
    char left = s.charAt(slow);
    // 左边界字符的计数减1
    map.put(left, map.get(left) - 1);
    // 如果计数为0，说明窗口内已无该字符，从map中移除
    if(map.get(left) == 0){
        map.remove(left);
    }
    // 左指针右移，缩小窗口
    slow++;
}
```

- 这里用 **while** 而非 **if**: 因为可能一次移除左边界字符后，窗口内不同字符数仍>2（比如窗口是 "abcc", 移除第一个 'a' 后还是 "bcc", 但如果窗口是 "abc", 移除 'a' 后就满足条件了）；
- 收缩逻辑: 先减少左边界字符的计数，若计数为0则从map中删除，再右移左指针，直到窗口内不同字符数≤2。

步骤5: 更新最优解

```
// 每次扩展/收缩后，计算当前窗口长度，更新最大值
maxLen = Math.max(maxLen, fast - slow + 1);
}
return maxLen;
```

窗口长度 = **fast - slow + 1**（因为指针是索引，包含两端），每次都比较并保留最大值。

3. 示例推演（以 s = "eceba" 为例）

fast	字符	map状态	map.size()	是否收缩	slow	窗口范围	窗口长度	maxLen
0	e	{e:1}	1	否	0	[0,0]	1	1

fast	字符	map状态	map.size()	是否收缩	slow	窗口范围	窗口长度	maxLen
1	c	{e:1, c:1}	2	否	0	[0,1]	2	2
2	e	{e:2, c:1}	2	否	0	[0,2]	3	3
3	b	{e:2, c:1, b:1}	3	是	0→1	[1,3]	3	3
		收缩后: {c:1, b:1}	2	否	1			
4	a	{c:1, b:1, a:1}	3	是	1→2	[2,4]	3	3
		收缩后: {b:1, a:1}	2	否	2			

最终返回 `maxLen = 3`，符合预期。

二、滑动窗口的通用逻辑

滑动窗口主要用于解决**子串/子数组的最值问题**（比如最长/最短子串、最多/最少字符等），核心逻辑可总结为：

1. 适用场景

- 问题要求找**连续**的子串/子数组；
- 条件与子串内的元素特征相关（如字符种类、元素和、元素个数等）；
- 暴力解法时间复杂度高（ $O(n^2)$ ），需要优化到 $O(n)$ 。

2. 通用步骤

1. **初始化：**
 - 左指针 `left = 0`，结果变量（如`maxLen`）；
 - 辅助数据结构（如哈希表、数组）：记录窗口内元素的状态（如计数、和）。
2. **扩展右边界：**
 - 用 `for` 循环让右指针 `right` 遍历整个数组/字符串；
 - 将当前元素加入窗口，更新辅助数据结构。
3. **收缩左边界：**
 - 当窗口不满足条件时（如字符数超限制、和超过目标），用 `while` 循环收缩左边界；
 - 移除左边界元素，更新辅助数据结构，左指针右移。
4. **更新结果：**
 - 每次扩展/收缩后，计算当前窗口的结果（如长度、和），更新最优解。

3. 关键注意点

- 用 `while` 收缩左边界：确保窗口最终满足条件（避免一次收缩不彻底）；
- 辅助数据结构的选择：字符类问题常用哈希表/数组（ASCII字符可用数组更高效），数值类问题可直接记录和；
- 窗口长度计算：`right - left + 1`（包含两端）。

总结

1. 滑动窗口是**双指针**的进阶，核心是通过动态调整左右边界，在一次遍历中找到最优解，时间复杂度 $O(n)$ ；
2. 解题四步：初始化→扩展右边界→收缩左边界→更新结果，其中**收缩条件**和**辅助数据结构**是核心；
3. 本题中，哈希表记录字符计数，收缩条件是`map.size()>2`，最终通过比较窗口长度得到最长子串长度。